



Algorithms and Complexity 2026

Mastery Test 2: Complexity

The mastery test is a mandatory and graded part of the course. The test consists of four tasks of varying difficulties. **Note that the tasks have different roles and your grade is not just a matter of number of problems solved! Make sure you understand from the grading table below which problems you need to solve in order to get the grade you want.**

Written solutions are submitted through Canvas and are *due on Tuesday, Apr 21, 19:00*. The written report should be in English and the file must be in pdf format.

If you have any questions about any of the problems, send an e-mail to algokomp-help@kth.se. Good luck!

Grading

As described in the detailed grading criteria on Canvas, the requirements for the overall grade on this mastery test are as follows.

Grade	Requirement
A	Pass problems 2, 3, and 4. Have rigorous correctness proofs on problems 2 and 3.
B	Like A but one of the following: (i) miss problem 4 or (ii) only one rigorous correctness proof or (iii) problem 1 instead of problem 2.
C	Pass problem 3, and either problem 1 or 2, and have at least one rigorous correctness proof.
D	Like C but one of the following: (i) miss problem 3, or (ii) no rigorous correctness proof.
E	Pass either problem 1 or problem 2 (one of the NP-completeness problems).
Fx	Miss one of the requirements for one of the problems for grade E.

(Note that for the “rigorous correctness proof” requirements, only problems 1, 2 and 3 are applicable.)

Mastery Test 2 Rules

- **The test must be solved individually. No discussion is allowed.** You are not allowed to discuss the problems with anyone except teachers in the course until *after the oral presentations* are completed.
- **Using external resources:** While not needed to solve the problems, you may consult sources outside the course material (e.g., lecture notes from a different course, or Wikipedia), provided you clearly document what you used. However you cannot freely take algorithms or results from outside the course material for granted – you must explain and prove such algorithms and results yourself “from scratch” in your solution, in your own words. This includes generative AI, subject to the additional specific rules below. Results from the course materials (book or lecture slides) can simply be cited and used directly without further explanation.
- **Document resources used:** Start your document with a short statement regarding what external resources (if any) you used to solve the problems, including whether you used generative AI. If any other sources apart from the course material are used as described in the previous item, these need to be clearly documented and cited. Even if you did not use any AI or other external sources, you still need to include a short statement explaining this.
- **Length limits:** Your submissions can have at most 1 500 words of text per problem, including the pseudo code. A solution for a problem that clearly exceeds this limit will not be graded. As a rule of thumb, 1 500 words usually corresponds to approximately 2-3 pages of text with normal typesetting.

Warning: If you use **Overleaf**, be aware that its word count function tends to significantly underestimate the number of words. So if Overleaf reports a word count approaching the limit (say, 1 250 words or more) then you should double-check the word count using some other tool to verify your count.

Specific rules regarding using AI

Using generative AI is allowed subject to the following rules.

- In order for everyone to have equal opportunity, the only allowed AI service is the Microsoft Copilot service provided by KTH (<https://m365.cloud.microsoft/chat/>, log in with your KTH login using <your-KTH-username>@ug.kth.se as the e-mail address).
- If you used AI in the process of solving the problems or creating your writeup, you must submit an appendix documenting *how* this was done by including copies of your chat logs, e.g. screenshots (if answers to some queries are excessively long they do not have to be included in full but can be truncated). This appendix can be added as a comment to your submission on Canvas up to 48 hours after the submission deadline – if you are working right up until the deadline then you do not need to take time away from the problems to do this documentation.
- Regardless of any AI assistance you may have obtained along the way, you are solely responsible for the contents of your submission. At the oral presentation, you need to be able to demonstrate a complete understanding of every aspect of your written solutions (without access to AI or any other 3rd party source).

Problems

1 Course Planning (NP-completeness, easier)

(This problem is a variant of problem 1 from Mastery Test 1; the difference is that there is now a variable number of courses whereas in the original problem there were only 5 courses.)

This semester Osquar is taking p different courses and there are in total n different homeworks across these courses. For simplicity we number these 1 to p and 1 to n respectively (rather than their actual codes/names like “DD2352” and “MAS1”).

Each homework $i \in \{1, \dots, n\}$ has a start day s_i when it is published and deadline day $d_i \geq s_i$ when it is due, as well as an identifier $c_i \in \{1, \dots, p\}$ indicating which course the homework belongs to. Somewhat unrealistically, all homeworks take the same amount of time to complete, and Osquar has the capacity to do up to some number f (an integer) of homeworks in one day.

If Osquar misses a homework i , i.e., if he does not do it during one of the days between s_i and d_i (inclusive), then he fails the course c_i that the homework belongs to. Some courses might not have any homeworks and Osquar automatically passes them no matter what he does.

Naturally Osquar would like to fail as few courses as possible. In the decision version of this, which we refer to as the COURSE PLANNING problem, an instance consists of course data as described above and a target number of courses t . The answer to an input is YES if there is a set of at least t many courses that can be passed, and NO otherwise.

Prove that the COURSE PLANNING problem is NP-complete.

You may use a reduction involving any of the following known NP-hard problems which were discussed in lectures, exercises and the book, listed here in alphabetical order: 3-COLORING, 3-DIMENSIONAL MATCHING, 3-SAT, (CNF-)SAT, DIRECTED/UNDIRECTED HAMILTONIAN CYCLE/PATH, GRAPH COLORING, INDEPENDENT SET, KNAPSACK, LOAD BALANCING, SET COVER, SUBSET SUM, TRAVELLING SALESMAN, VERTEX COVER.

Requirements for problem 1:

See next page for the requirements for problem 1.

2 Program Design (NP-completeness, harder)

In this problem we consider the complicated task of designing a study program. More concretely, we consider a very simplistic setting according to the following:

- The study program should be s semesters long.
- There is a given set of n courses that should be included in the program (each having the same number of credits and running for a single semester).
- Each semester the students should take exactly n/s many courses. (It can be assumed that n is evenly divisible by s so that this is an integer.)
- Some courses are prerequisites of others. If course i is a prerequisite for course j then course i needs to be taken in an earlier semester than course j . (The prerequisites are of course acyclic.)

Given this data (the parameters s and n and the description of the prerequisites of the n courses), the PROGRAM DESIGN problem asks whether it is possible to create a study program consisting of these n courses, while having exactly n/s courses per semester and adhering to the prerequisites.

Prove that the PROGRAM DESIGN problem is NP-complete. You may use a reduction involving any of the known NP-hard problems listed in the previous problem.

Requirements for problems 1 and 2:

The following requirements apply for both problems 1 and 2. An item labelled “mostly” indicates that this needs to be present, but that there is room to correct minor errors or issues in the oral presentation.

Component	Requirement for written solutions	Requirement for oral presentation
General requirements fulfilled (declaration of sources, at most 1500 words including pseudo code, etc)	Yes	–
Show problem is in NP	Mostly	Yes
Clear description of reduction, with some combination of text/figures/pseudo code	Yes	Yes
Reduction is sufficiently efficient	Yes	–
Reduction is correct, establishes NP-hardness	Yes	–
Correct analysis of time complexity	Mostly	Yes
Explanation of general ideas for why reduction is correct	Mostly	Yes
Complete rigorous correctness proof	Mostly (but for higher grades only)	Yes (but for higher grades only)

3 Tulips (Decidability)

You are playing a video game where the goal is to build and manage a business selling tulips. To control the business you have to make n strategic choices (like setting the selling price for each tulip type, the salary for each employee category, and so on). Given these choices $x_1, \dots, x_n$ (e.g. x_1 = price of Queen of Night tulips, x_2 = price of Black Parrot tulips, etc), you earn some profit $P(x_1, \dots, x_n)$ in the game. Dedicated fans of the game have completely reverse-engineered the profit function P , which turns out to be a polynomial with integer coefficients in the n variables. All the choices $x_1, \dots, x_n$ are numeric and can only be assigned positive integer values. But somewhat surprisingly there is no upper bound on how large these can be; you can make them as large as you like.



Now you start wondering, what is the maximum profit that can be achieved by a well-designed strategy? Can the profit be made arbitrarily large or is there some maximum possible achievable profit?

Formally, in the TULIPS problem, an instance consists of an multivariate polynomial $P : \mathbb{Z}^n \rightarrow \mathbb{Z}$ in n variables. The answer for the instance P is NO if there exists a finite “profit cap” $C \in \mathbb{Z}$ such that $P(x_1, \dots, x_n) \leq C$ for all positive integers $x_1, \dots, x_n$. Otherwise, if no such C exists, the answer is YES.

For example, if $P(x_1, x_2) = 4x_1^2x_2 - x_1^4 - 35x_2$ then the answer is YES, since this function can easily be made arbitrarily large (e.g. setting $x_1 = 3$ and letting $x_2 \rightarrow \infty$). On the other hand if $P(x_1, x_2, x_3) = 2026 - 300x_1^5 - 85x_2 - 4x_3^9$ then the answer is NO, since $P(x_1, x_2, x_3) \leq 2026$ for all positive x_1, x_2, x_3 . In fact the maximum possible profit for this polynomial is $P(1, 1, 1) = 2026 - 300 \cdot 1^5 - 85 \cdot 1 - 4 \cdot 1^9 = 1637$; but note that in NO cases like this one we do not need to find the precise maximum possible profit, just establish that a finite upper bound exists.

Is this problem decidable or undecidable? If decidable, prove it by giving an algorithm. If undecidable, prove it either by an appropriate reduction or by a direct proof.

Requirements for problem 3:

The requirements for problem 3 are as follows. An item labelled “mostly” indicates that this needs to be present, but that there is room to correct minor errors or issues in the oral presentation.

Component	Requirement for written solutions	Requirement for oral presentation
General requirements fulfilled (declaration of sources, at most 1500 words including pseudo code, etc)	Yes	–
Clear description of construction (whether an algorithm or undecidability proof)	Yes	Yes
The described construction works (can be used to establish decidability/undecidability)	Yes	–
Explanation of general ideas for why construction works	Mostly	Yes
Complete rigorous proof that construction works	Mostly (but for higher grades only)	Yes (but for higher grades only)

4 Program Search (Decision vs Search)

In this problem we revisit the PROGRAM DESIGN problem from Problem 2 (NP-completeness, harder) above (it is by no means necessary to solve that problem to be able to solve this one). The *search* version of this problem is, given an instance (n, s, R) (where R describes the prerequisites between courses), to produce a valid study program on these n courses satisfying the requirements described in Problem 2.

Suppose we have a hypothetical algorithm $\text{DecideProgram}(n, s, R)$ for the decision version (i.e., the problem defined in Problem 2) that runs in $O(T(n))$ time for some function $T(n)$.

Using DecideProgram as a black box, give an algorithm for the search version of the problem that runs in $O(n^3 \cdot T(n) + n^3)$ time.

Requirements for problem 4:

The requirements for problem 4 are as follows. An item labelled “mostly” indicates that this needs to be present, but that there is room to correct minor errors or issues in the oral presentation.

Component	Requirement for written solutions	Requirement for oral presentation
General requirements fulfilled (declaration of sources, at most 1500 words including pseudo code, etc)	Yes	–
Clear description of algorithm, with some combination of text/figures/pseudo code	Yes	Yes
Algorithm is correct	Yes	–
Algorithm is efficient enough, satisfies time complexity requirement	Yes	–
Correct analysis of time complexity	Mostly	Yes
Explanation of general ideas for why algorithm is correct	Mostly	Yes

(Note: no formal proof of correctness is needed in this problem, even for higher grades, but you still need to give a convincing informal explanation of why your algorithm is correct.)